



به نام خالق ستارگان
دانشگاه تهران
دانشکده مهندسی برق و
کامپیوتر



تمرین هفتم درس جداسازی کور منابع (BSS)

دکتر سعید اخوان

نام و نام خانوادگی	سیده غزل موسوی
شماره دانشجویی	۸۱۰۱۰۰۲۵۹
مهلت ارسال پاسخ	۱۴۰۴.۲.۳

روش‌های مختلف بازیابی سیگنال sparse

الف) در روش subset selection ما تمام جایگشت‌های N_0 تایی از تعداد منابع را بررسی می‌کنیم و خطا آن را محاسبه کرده و در برداری ذخیره می‌کنیم سپس جایگشتی که منجر به کمترین خطا شده است را به عنوان درایه‌های غیر صفر منابع در نظر می‌گیریم و مقدار هر درایه نیز از رابطه زیر به دست می‌آوریم.

$$error = \|D\hat{s} - x\|_2$$

$$\hat{s} = D^\dagger x$$

```
=====
Subst Selection:
Runtime: 3.671e-01
Subset selection error is: 4.24e-15
-----
      Index |      S_hat (Value)
-----
          3 |          5.000000
          6 |          7.000000
         54 |         -3.000000
-----
=====
```

شکل ۱.۱. تاج روش Subset Selection

دانستن N_0 در این بخش به جست و جو کمتر منجر می‌شود زیرا اگر N_0 را نمی‌دانستیم باید برای حالت‌های مختلف آن تمامی جایگشت‌های ممکن را می‌گشتیم که تعداد آن بسیار زیاد می‌شد.

ب) در این بخش مسئله بهینه‌سازی را به شکل زیر تغییر می‌دهیم و وقتی به جای نرم صفر (تعداد درایه‌های غیرصفر) نرم دو s را در نظر می‌گیریم جوابی که به دست می‌آید لزوماً sparse نیست و به همین دلیل این روش خیلی روش مناسبی نیست.

$$\begin{cases} \hat{s} = \operatorname{argmin} \|s\|_2 \\ s.t. \quad x = Ds \end{cases}$$

ج) در روش Matching Pursuit (MP) به این صورت عمل می‌کنیم که تصویر ماتریس مشاهدات را بر روی هر بردار اتم به دست می‌آوریم و ماکسیمم مقدار تصویر و اندیس بردار اتم را به عنوان مقدار و اندیس ماتریس منابع در نظر می‌گیریم. سپس تصویر مشاهدات روی بردار اتم مورد نظر را، از آن کم کرده و مجدداً روش بالا را N_0 بار انجام می‌دهیم.

در نتیجه اگر N_0 را بدانیم که تعداد تکرار مشخص است اگر نه باید از $N_0 = 1$ شروع کنیم و همینطور ادامه دهیم تا به خطای مطلوب مان برسیم.

```
=====
Matching Pursuit Kown N0 = 3:
Runtime: 2.300e-03
Error is: 2.45e+00
-----
      Index |      S_hat (Value)
-----
          6 |      11.179061
         60 |      -3.755818
          9 |      -2.053218
-----
=====
```

شکل ۲. نتایج روش MP با فرض دانستن $N_0 = 3$

```
=====
Matching Pursuit
Sparsity Level is: N0 = 7.
Runtime: 6.460e-05
Error is: 4.52e-01
-----
      Index |      S_hat (Value)
-----
          6 |      11.179061
         60 |      -3.755818
          9 |      -2.053218
         30 |      -1.885734
         16 |       1.226286
         15 |       0.628608
          2 |       0.588767
-----
=====
```

شکل ۳. نتایج روش MP بدون دانستن N_0

د) روش Orthogonal Matching Pursuit مشابه روش قبل است با این تفاوت که مقادیر ماتریس منابع با تمامی بردارهای

اتم تا به آن لحظه پیدا شده به روزرسانی می‌شوند، این روش دقت بیشتری دارد و سبب می‌شود مثل روش قبل جست و جو

حریصانه نداشته باشیم و مقادیر ماتریس منابع بهتر تخمین زده شود.

```
=====
Orthogonal Matching Pursuit Kown N0 = 3
Runtime: 2.445e-03
Error is: 2.17e+00
-----
      Index |      S_hat(Value)
-----
          6 |          9.954504
         60 |         -4.142751
         16 |          2.507518
-----
=====
```

شکل ۴. نتایج روش OMP با فرض دانستن $N_0 = 3$

```
=====
Orthogonal Matching Pursuit
Sparsity Level is: N0 = 6.
Runtime: 1.810e-04
Error is: 4.41e-01
-----
      Index |      S_hat(Value)
-----
          6 |          9.486019
         60 |         -3.770273
         16 |          2.574815
         15 |          1.985342
         56 |          0.807966
         59 |          0.564686
-----
=====
```

شکل ۵. نتایج روش OMP بدون دانستن N_0

ه) در روش Basis Pursuit(BP) به جای نرم صفر نرم یک قرار می‌دهیم و مسئله بهینه‌سازی به فرم زیر تغییر می‌کند:

$$\begin{cases} \hat{s} = \operatorname{argmin} \|s\|_1 \\ \text{s.t. } x = Ds \end{cases}$$

با ساده‌سازی و یکسری تعاریف مسئله به شکل زیر تبدیل می‌شود که یک مسئله معروف به نام Linear Programming است و

دستور آماده آن در متلب linprog است. دانستن مقدار N_0 کمکی نمی‌کند.

$$\begin{cases} \hat{s} = \operatorname{argmin} \sum_{i=1}^{2N} y_i \\ \text{s.t. } x = D_1 s \\ D_1 = [D, -D] \\ y = \begin{bmatrix} S_n^+ \\ S_n^- \end{bmatrix} \end{cases}$$

```
=====
Basis Pursuit:
Runtime: 8.116e-03
Error is: 4.83e-15
-----
      Index |      S_hat(Value)
-----
          3 |          5.000000
          6 |          7.000000
         54 |         -3.000000
-----
=====
```

شکل ۶. نتایج روش BP

و) در روش IRLS مسئله بهینه سازی به شکل زیر تبدیل می شود و در ابتدا بردار وزن W را به صورت رندوم تعریف می کنیم و در هر مرحله ماتریس منابع را با استفاده از آن تخمین می زنیم. و سپس W را نیز با توجه به ماتریس منابع به روزرسانی می کنیم این کار را تکرار می کنیم در نهایت مقادیری از W که بیشتر هستند S کمتری دارند.

$$\begin{cases} \hat{s} = \underset{s}{\operatorname{argmin}} \|s\|_1 = \sum_{n=1}^N |S_n| = \sum_{n=1}^N \frac{1}{|S_n|} |S_n|^2 = \sum_{n=1}^N w_n |S_n|^2 = \\ s.t. \quad x = Ds \\ S = (w^{-1}D^T)(Dw^{-1}D^T)^{-1}x \end{cases}$$

```
=====
```

IRLS:

Runtime: 1.951e-03

Error is: 1.10e-10

```
-----
```

Index	S_hat (Value)
3	4.984313
6	6.986304
54	-2.983566

```
-----
```

شکل ۷. نتایج بخش IRLS

ی) روش اول روش دقیقی است، ولی چون همه حالات را جست و جو می کند روش زمانبری است و اصلا بهینه نیست خصوصا وقتی N_0 زیاد است و مقدار آن را نمی دانیم اما خطای کمی دارد و درست تخمین می زند.

روش MP نسبت به روش قبل سریع تر است ولی باز هم خیلی دقیق نیست چون فقط نزدیک ترین را می بیند و مقدار را فقط براسای همان تعیین می کند خیلی دقیق نیست

روش OMP زمان بیشتری نیاز دارد ولی چون مقادیر منابع را آپدیت می کند مقداری دقیق تر است.

روش BS روش خوبی است هم از نظر زمانی هم از نظر دقت و به نظر می رسد از بقیه روش ها بهتر عمل کرده است.

روش ILRS زمان بر است ولی دقت و جواب مطلوبی را ارائه می دهد.